

Ricorsione

Table of contents

La Ricorsione	1
Esempio: il problema degli 11 cammelli.	1
Definizione	9
Tipi di ricorsione - casi principali	9
Ricorsione lineare (diretta)	9
Come funziona lo stack delle chiamate	10
Ricorsione multipla	11
Fibonacci iterativo in Python	12
Ricorsione mutua o indiretta	12
Ricorsione vs iterazione	13
Limiti pratici dello stack	14
Next step	14
Esercizi	15
Esercizi	16

La Ricorsione

Esempio: il problema degli 11 cammelli.

Un mercante di cammelli lascia ai suoi tre figli i suoi 11 cammelli, stabilendo che la ripartizione dell'asse ereditario sarà la seguente

- primo figlio -> $1/2$
- secondo figlio -> $1/4$
- terzo figlio -> $1/6$

Al primo figlio spettano 5.5 cammelli, al secondo figlio 2.75 cammelli, al terzo figlio circa 1.83 cammelli. I figli non vorrebbero uccidere i cammelli. Allora un vecchio saggio presta loro il proprio cammello, portando il totale a 12.

In questo modo a ciascun figlio spettano rispettivamente 6, 3 e 2 cammelli. Nessun cammello viene ucciso, ogni figlio ha una quota maggiore rispetto a quella che pensava di ottenere, ed il saggio può riprendere il proprio cammello.

Come è possibile?

Il testamento non ripartisce l'intera eredità. Sommando le quote otteniamo $1/2 + 1/4 + 1/6 = 11/12$, abbiamo quindi una **quota indivisa** pari a $\frac{1}{12}$.

Il testamento si può interpretare nel senso che la quota indivisa va perduta (per esempio in beneficenza), oppure che **la quota indivisa va ripartita secondo le percentuali dette**.

Seguendo la seconda interpretazione, qual è l'eredità di ciascun figlio? **Occorre riapplicare la stessa ripartizione alla rimanenza.**

Esaminiamo le ripartizioni dal punto di vista del figlio 1, a cui spetta $\frac{1}{2}$ dell'asse ereditario (11 cammelli)

$$\underbrace{11} \cdot \boxed{\frac{1}{2}}$$

Esaminiamo le ripartizioni dal punto di vista del figlio 1, a cui spetta $\frac{1}{2}$ dell'asse ereditario (11 cammelli)

$$11 \cdot \boxed{\frac{1}{2}} + \underbrace{\frac{11}{12}} \cdot \boxed{\frac{1}{2}}$$

Ora si distribuisce la rimanenza di $\frac{11}{12}$

Esaminiamo le ripartizioni dal punto di vista del figlio 1, a cui spetta $\frac{1}{2}$ dell'asse ereditario (11 cammelli)

$$11 \cdot \boxed{\frac{1}{2}} + \frac{11}{12} \cdot \boxed{\frac{1}{2}} + \underbrace{\frac{11}{12^2}} \cdot \boxed{\frac{1}{2}}$$

Ora si distribuisce la rimanenza di $\frac{11}{12^2}$

Esaminiamo le ripartizioni dal punto di vista del figlio 1, a cui spetta $\frac{1}{2}$ dell'asse ereditario (11 cammelli)

$$11 \cdot \boxed{\frac{1}{2}} + \frac{11}{12} \cdot \boxed{\frac{1}{2}} + \frac{11}{12^2} \cdot \boxed{\frac{1}{2}} + \frac{11}{\underbrace{12^3}} \cdot \boxed{\frac{1}{2}}$$

Ora si distribuisce la rimanenza di $\frac{11}{12^3} \dots$

Esaminiamo le ripartizioni dal punto di vista del figlio 1, a cui spetta $\frac{1}{2}$ dell'asse ereditario (11 cammelli)

$$\begin{aligned} & 11 \cdot \boxed{\frac{1}{2}} + \frac{11}{12} \cdot \boxed{\frac{1}{2}} + \frac{11}{12^2} \cdot \boxed{\frac{1}{2}} + \frac{11}{12^3} \cdot \boxed{\frac{1}{2}} + \dots = \\ & = 11 \cdot \boxed{\frac{1}{2}} \cdot \left(1 + \frac{1}{12} + \frac{1}{12^2} + \frac{1}{12^3} + \dots \right) = 11 \cdot \boxed{\frac{1}{2}} \cdot \sum_{k=0}^{\infty} \left(\frac{1}{12} \right)^k \end{aligned}$$

La serie geometrica vale:

$$\sum_{k=0}^{\infty} \left(\frac{1}{12} \right)^k = \frac{1}{1 - \frac{1}{12}} = \frac{12}{11}$$

Quindi effettivamente la quota del figlio 1 converge a

$$11 \cdot \boxed{\frac{1}{2}} \cdot \frac{12}{11} = 6$$

Lo stesso ragionamento vale per gli altri figli:

Quota figlio 2:

$$11 \cdot \boxed{\frac{1}{4}} \cdot \sum_{k=0}^{\infty} \left(\frac{1}{12} \right)^k = 11 \cdot \boxed{\frac{1}{4}} \cdot \frac{12}{11} = 3$$

Quota figlio 3:

$$= 11 \cdot \boxed{\frac{1}{6}} \cdot \sum_{k=0}^{\infty} \left(\frac{1}{12} \right)^k = 11 \cdot \boxed{\frac{1}{6}} \cdot \frac{12}{11} = 2$$

Calcolo l'eredità spettante al primo figlio e la quota indivisa

```
def eredita_figlio1(asse_ereditario):
    # percentuali spettanti
    quota_percentuale_f1 = 1/2
    quota_percentuale_f2 = 1/4
    quota_percentuale_f3 = 1/6

    # quote assegnate
    quota_f1 = asse_ereditario * quota_percentuale_f1
    quota_f2 = asse_ereditario * quota_percentuale_f2
    quota_f3 = asse_ereditario * quota_percentuale_f3

    # residuo non assegnato
    residuo = asse_ereditario - (quota_f1 + quota_f2 + quota_f3)

    return quota_f1, residuo
#---

quota_f1, residuo = eredita_figlio1(11)
print(f"quota figlio 1: {quota_f1:.3f}  residuo: {residuo:.3f}")
```

quota figlio 1: 5.500 residuo: 0.917

Ora posso usare questa funzione per calcolare la quota totale spettante al figlio 1.

Calcolo la 'quota aggiuntiva' ottenuta dalla ripartizione del residuo, e la sommo alla quota iniziale.

```
quota_aggiuntiva_f1, residuo = eredita_figlio1(residuo)
quota_f1 = quota_f1 + quota_aggiuntiva_f1
print(f"quota figlio 1: {quota_f1:.3f}  residuo: {residuo:.3f}")

quota_aggiuntiva_f1, residuo = eredita_figlio1(residuo)
quota_f1 = quota_f1 + quota_aggiuntiva_f1
print(f"quota figlio 1: {quota_f1:.3f}  residuo: {residuo:.3f}")

quota_aggiuntiva_f1, residuo = eredita_figlio1(residuo)
quota_f1 = quota_f1 + quota_aggiuntiva_f1
print(f"quota figlio 1: {quota_f1:.3f}  residuo: {residuo:.3f}")
```

```
quota figlio 1: 5.958  residuo: 0.076
quota figlio 1: 5.997  residuo: 0.006
quota figlio 1: 6.000  residuo: 0.001
```

In questo esempio la funzione viene richiamata più volte dal programma principale. Ogni volta prende il **residuo** dell'asse ereditario e ne ricalcola la quota spettante al figlio 1.

Invece di richiamare manualmente la funzione molte volte, posso far sì che sia **la funzione stessa a richiamare se stessa**. Questa tecnica si chiama **ricorsione**.

La ricorsione termina quando il residuo diventa abbastanza piccolo da poter essere considerato nullo.

```
def eredita_figlio1(asse_ereditario):
    print('.') # stampa un punto ad ogni iterazione
    soglia=0.001
    if asse_ereditario < soglia:
        return 0, asse_ereditario
    # percentuali spettanti
    quota_percentuale_f1 = 1/2
    quota_percentuale_f2 = 1/4
    quota_percentuale_f3 = 1/6

    # quote assegnate
    quota_f1 = asse_ereditario * quota_percentuale_f1
    quota_f2 = asse_ereditario * quota_percentuale_f2
    quota_f3 = asse_ereditario * quota_percentuale_f3

    # residuo non assegnato
    residuo = asse_ereditario - (quota_f1 + quota_f2 + quota_f3)

    # ricorsione
    quota_aggiuntiva_f1, residuo = eredita_figlio1(residuo)

    # aggiungo alla quota corrente (quota_f1)
    # la quota proveniente dalla ricorsione
    quota_f1 = quota_f1 + quota_aggiuntiva_f1

    return quota_f1, residuo
#---
```

```
quota_f1, residuo = eredita_figlio1(11)
print(f"quota figlio 1: {quota_f1:.3f}  residuo: {residuo:.3f}")
```

Miglioramenti:

- calcolo le quote per tutti e tre i figli

```
def eredita(asse_ereditario):
    print('.') # stampa un punto ad ogni iterazione
    soglia=0.001
    if asse_ereditario < soglia:
        return 0,0,0, asse_ereditario
    # percentuali spettanti
    quota_percentuale_f1 = 1/2
    quota_percentuale_f2 = 1/4
    quota_percentuale_f3 = 1/6

    # quote assegnate
    quota_f1 = asse_ereditario * quota_percentuale_f1
    quota_f2 = asse_ereditario * quota_percentuale_f2
    quota_f3 = asse_ereditario * quota_percentuale_f3

    # residuo non assegnato
    residuo = asse_ereditario - (quota_f1 + quota_f2 + quota_f3)

    # ricorsione
    (
        quota_aggiuntiva_f1,
        quota_aggiuntiva_f2,
        quota_aggiuntiva_f3,
        residuo,
    ) = eredita(residuo)
    # aggiungo alla quota corrente
    # la quota proveniente dalla ricorsione
    quota_f1 = quota_f1 + quota_aggiuntiva_f1
    quota_f2 = quota_f2 + quota_aggiuntiva_f2
    quota_f3 = quota_f3 + quota_aggiuntiva_f3

    return quota_f1, quota_f2, quota_f3, residuo
```

```
#---

quota_f1, quota_f2, quota_f3, residuo = eredita(11)
print(f"quota figlio 1: {quota_f1:.3f} ")
print(f"quota figlio 2: {quota_f2:.3f} ")
print(f"quota figlio 3: {quota_f3:.3f} ")

print(f"residuo: {residuo:.3f} ")
```

Miglioramenti:

- quote percentuali e soglia diventano parametri. La soglia ha un valore di default

```
def eredita(asse_ereditario, p_f1,p_f2,p_f3, soglia=0.001):
    # p_f{i}-> quote percentuali spettanti

    print('.') # stampa un punto ad ogni iterazione
    if asse_ereditario < soglia:
        return 0,0,0, asse_ereditario

    # quote assegnate
    quota_f1 = asse_ereditario * p_f1
    quota_f2 = asse_ereditario * p_f2
    quota_f3 = asse_ereditario * p_f3

    # residuo non assegnato
    residuo = asse_ereditario - (quota_f1 + quota_f2 + quota_f3)

    # ricorsione
    (
        quota_aggiuntiva_f1,
        quota_aggiuntiva_f2,
        quota_aggiuntiva_f3,
        residuo,
    ) = eredita(residuo, p_f1, p_f2, p_f3, soglia)
    # aggiungo alla quota corrente
    # la quota proveniente dalla ricorsione
    quota_f1 = quota_f1 + quota_aggiuntiva_f1
    quota_f2 = quota_f2 + quota_aggiuntiva_f2
    quota_f3 = quota_f3 + quota_aggiuntiva_f3
```

```

    return quota_f1, quota_f2, quota_f3, residuo
#---

quota_f1, quota_f2, quota_f3, residuo = eredita(11, 1/2, 1/4, 1/6)
print(f"quota figlio 1: {quota_f1:.3f} ")
print(f"quota figlio 2: {quota_f2:.3f} ")
print(f"quota figlio 3: {quota_f3:.3f} ")

print(f"residuo: {residuo:.3f} ")

```

Miglioramenti:

- generalizzo a k figli

```

def eredita(asse_ereditario, perc_figli, soglia=0.001):
    n_figli = len(perc_figli)

    print('.') # stampa un punto a ogni iterazione

    if asse_ereditario < soglia:
        # nota: risultato_ricorsivo
        # contiene sia le quote
        # sia il residuo
        risultato_ricorsivo = [0] * n_figli
        risultato_ricorsivo.append(asse_ereditario)
        return risultato_ricorsivo

    quote = [0] * n_figli

    for i in range(n_figli):
        quote[i] = asse_ereditario * perc_figli[i]

    assegnato = 0
    for q in quote:
        assegnato += q

    residuo = asse_ereditario - assegnato

    risultato_ricorsivo = eredita(residuo, perc_figli, soglia)

    for i in range(n_figli):
        quote[i] += risultato_ricorsivo[i]

```

```

    return quote + [risultato_ricorsivo[-1]]
#---

quota_f1, quota_f2, quota_f3, residuo = eredita(11, [1/2, 1/4, 1/6])
print(f"quota figlio 1: {quota_f1:.3f} ")
print(f"quota figlio 2: {quota_f2:.3f} ")
print(f"quota figlio 3: {quota_f3:.3f} ")

print(f"residuo: {residuo:.3f} ")

```

```

quota figlio 1: 6.000
quota figlio 2: 3.000
quota figlio 3: 2.000

```

Definizione

Una funzione è **ricorsiva** se richiama se stessa direttamente o indirettamente.

Ogni definizione ricorsiva richiede:

1. **Caso base** — condizione che termina la ricorsione (non genera nuove chiamate).
2. **Caso ricorsivo** — il problema viene ridotto a una versione più piccola dello stesso problema. .

Senza caso base la ricorsione non termina portando ad una saturazione della memoria (*stack overflow*).



Tipi di ricorsione - casi principali

Ricorsione lineare (diretta)

La funzione chiama se stessa **una sola volta** per esecuzione.

Esempio: fattoriale

$$n! = \begin{cases} n \cdot (n-1)! & \text{se } n > 1 \\ 1 & \text{se } n = 0 \text{ o } n = 1 \end{cases}$$

Fattoriale — C

```
int fattoriale(int n) {
    if (n <= 1) return 1;           // caso base
    return n * fattoriale(n - 1);  // caso ricorsivo
}
```

Fattoriale — Python

```
# NB: annotazioni di tipo opzionali
def fattoriale(n: int) -> int:
    if n <= 1:                # caso base
        return 1
    return n * fattoriale(n - 1)
```

ESECUZIONE

```
fattoriale(5)
= 5 * fattoriale(4)
= 5 * 4 * fattoriale(3)
= 5 * 4 * 3 * fattoriale(2)
= 5 * 4 * 3 * 2 * fattoriale(1)
= 5 * 4 * 3 * 2 * 1 * fattoriale(0)
= 5 * 4 * 3 * 2 * 1 * 1 =
= 120
```

Come funziona lo stack delle chiamate

Ogni chiamata a funzione crea un nuovo contesto di esecuzione.

Le chiamate ricorsive vengono memorizzate nello **stack delle chiamate**.

Con la ricorsione i frame si accumulano fino al caso base, poi vengono rimossi (unwinding) restituendo i valori verso il chiamante originale.

```
fattoriale(3)
    fattoriale(2)
        fattoriale(1)
            fattoriale(0) ← caso base: rende 1
        ← rende 1*1 = 1
    ← rende 2*1 = 2
← rende 3*2 = 6
```

Ricorsione multipla

La funzione chiama se stessa **più di una volta** per esecuzione. L'albero delle chiamate cresce esponenzialmente.

Esempio: sequenza d Fibonacci https://it.wikipedia.org/wiki/Successione_di_Fibonacci

$$F_n = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ F_{n-1} + F_{n-2} & \text{se } n > 1 \end{cases}$$

Sequenza:

[0, 1, 1, 2, 3, 5, 8, 13, 21, ...]

Questo è un esempio di algoritmo **non adatto a implementazione ricorsiva** perché ripete molte chiamate inutili.

Fibonacci — C

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Fibonacci — Python

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Chiamate:

```
fibonacci(5)  
  fibonacci(4)  
    fibonacci(3)  
      fibonacci(2)  
        fibonacci(3)
```

Come si vede, `fibonacci(3)` viene calcolato più volte. In questi casi una soluzione iterativa è spesso migliore.

Attenzione: `fibonacci(40)` genera ~2,5 miliardi di chiamate, mentre l'implementazione iterativa ha un costo di $O(n)$. Questo è un caso in cui la ricorsione **non** è la scelta ottima.

Fibonacci iterativo in Python

L'idea dell'algoritmo iterativo è mantenere sempre in memoria solo gli ultimi due numeri della sequenza.

```
def fibonacci_iterativo(n):
    previous = 0
    current = 1

    for i in range(n):
        next_value = previous + current

        previous = current
        current = next_value

    return previous
```

n	F_n (previous)	current	next_value
0	0	1	1
1	1	1	2
2	1	2	3
3	2	3	5
4	3	5	8
5	5	8	13
6	8	13	21
7	13	21	...

Ricorsione mutua o indiretta

Due o più funzioni si chiamano a vicenda.

$$f(n) = k_1 \cdot g(n-1)$$

$$g(n) = k_2 \cdot f(n-1)$$

Esempio

C

```
int is_even(int n);
int is_odd(int n);

int is_even(int n) {
    if (n == 0) return 1;
    return is_odd(n - 1);
}

int is_odd(int n) {
    if (n == 0) return 0;
    return is_even(n - 1);
}
```

Python

```
def is_even(n: int) -> bool:
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n: int) -> bool:
    if n == 0: return False
    return is_even(n - 1)
```

Esempi meno banali di ricorsione indiretta sono i parser o l'attraversamento dei grafi.

Ricorsione vs iterazione

Criterio	Ricorsione	Iterazione
Leggibilità	Alta su problemi naturalmente ricorsivi	Alta su problemi lineari
Stack	Usa lo stack di sistema (limite!)	Nessun overhead di stack
Efficienza	Overhead per ogni chiamata	In genere più veloce
Stato	Implicito nei frame	Esplicito nelle variabili

Regola pratica:

- Usare la ricorsione è comodo quando il problema ha struttura ricorsiva naturale (alberi, suddivisioni del dominio, backtracking), e quindi la formulazione dell'algoritmo è immediata con la ricorsione. Tuttavia spesso, per questioni di ottimizzazione si trasforma l'algoritmo ricorsivo in un algoritmo iterativo,
 - In alcuni casi è meglio usare direttamente un approccio iterativo (somme, ricerche sequenziali).
-
-

Limiti pratici dello stack

In C lo stack di default è tipicamente 1–8 MB.

In Python il limite di default è **1000 frame** (`sys.getrecursionlimit()`).

```
import sys
print(sys.getrecursionlimit()) # 1000
sys.setrecursionlimit(5000)    # aumentabile, ma con cautela
```

Per ricorsioni profonde su strutture dati (liste concatenate, alberi sbilanciati) occorre valutare alternative iterative o aumentare il limite consapevolmente.

Next step

La ricorsione è la base per comprendere:

- **Algoritmi divide et impera** (Merge Sort, Quick Sort)
 - **Deallocazione ricorsiva** di strutture dati dinamiche (linked list, alberi)
 - **Backtracking** (esplorazione di spazi di ricerca)
 - **Analisi della complessità computazionale** ricorsiva (relazioni di ricorrenza)
-

Esercizi

Implementare in modo ricorsivo e non ricorsivo

- Fattoriale
- https://en.wikipedia.org/wiki/Triangular_number
- Fibonacci
- Implementate in C la soluzione dell'esercizio dei cammelli
- Deallocare uno stack implementato mediante lista concatenata

Traccia:

```
void free_stack(Node* top) {  
    if (top == NULL) {  
        return;  
    }  
  
    free_stack(top->next);  
    free(top);  
}
```

- Deallocare una coda implementato mediante lista concatenata

```
void free_queue_nodes(Node* node) {  
    if (node == NULL) {  
        return;  
    }  
  
    free_queue_nodes(node->next);  
    free(node);  
}  
  
void free_queue(Queue* queue) {  
    free_queue_nodes(queue->front);  
    free(queue);  
}
```